

«Изборник»: что умеет система

Лабораторная работа № 3 · вариант 4 · автоматическое реферирование методом Sentence extraction + OSTIS (русский и немецкий; научные статьи по computer science и сочинения по литературе)

«Изборник» строит реферат документа объёмом около 10 страниц. Реферат состоит из двух разделов: классического реферата — самых весомых предложений документа в порядке текста — и реферата в виде списка ключевых слов с подчинёнными им словосочетаниями. Система работает с русскими и немецкими текстами двух предметных областей. Документы хранятся в базе знаний ostis-системы НИКА, а реферат строит sc-агент и записывает его туда же; если ostis-система не запущена, тот же алгоритм выполняется локально.

Зачем это нужно: по реферату за минуту понятно, о чём десятистраничный документ и стоит ли его читать. Это пригодится электронным библиотекам и репозиториям статей, поисковым системам для предпросмотра документов, а реферат в базе знаний сразу доступен другим агентам ostis-системы.

В системе шесть рабочих страниц. Кроме них есть «Справка» и паук Пафнутий в углу страницы — они ниже не описываются. Запуск: **python run.py**, адрес **http://127.0.0.1:8030/**; ostis-система — **docker compose up -d --no-build** в каталоге НИКА.

Страница	Адрес	Для чего
Коллекция	/	20 документов тестовой коллекции, выбор документа
Реферат	/doc/<документ>	оба раздела реферата, расчёт, сохранение и печать
Исходный документ	/doc/<документ>/source	полный текст с отмеченными предложениями реферата
Свой документ	/text	реферат вставленного текста или загруженного файла
Оценка	/evaluation	точность по мере ROUGE и затраченное время
OSTIS	/ostis	состояние ostis-системы, база знаний, sc-агент

1. Коллекция <http://127.0.0.1:8030/>

Три карточки: «Документы» (20, по 5 в каждой группе «язык × предметная область»), «Объём документа» и «OSTIS» — подключена ли ostis-система и кто строит рефераты. Ниже — четыре таблицы, по одной на группу: заголовок документа (ссылка на реферат) с авторами, объём в знаках и страницах, источник (ссылка на оригинал и лицензия), кнопка «Реферат» и ссылка «исходный документ». Внизу — документы, загруженные пользователем в этом сеансе.

Пример. Объём документов — от 16 415 до 18 759 знаков, то есть 9,1–10,4 страницы по 1 800 знаков. В группе «русский, CS» — статья Н. А. Гулякиной и И. Т. Давыденко «Семантические модели и метод согласованной разработки баз знаний» (КиберЛенинка, CC BY, 2020): 18 192 знака, 10,1 страницы.

Как это работает. Методичка требует документов одинакового размера — например, 10 страниц А4. Объём задан как 10 страниц машинописного текста по 1 800 знаков (18 000 знаков, допуск $\pm 10\%$). Русские научные статьи взяты с КиберЛенинки: у них есть авторская аннотация и ключевые слова, которые служат эталоном при оценке. Немецкие статьи по CS и литературоведческие статьи на обоих языках взяты из Википедии, эталон — вводный раздел статьи. Эталон из текста документа удалён, иначе система «находила» бы его и точность была бы завышена.

2. Реферат /doc/<документ>

Главная страница системы. Вверху — язык, предметная область, объём и активные ссылки: «Исходный документ», «Оригинал» (страница источника), «Текст файлом»; справа — кто построил реферат (sc-агент OSTIS или локальный расчёт) и за сколько. Панель инструментов: размер реферата (3–30 предложений, по умолчанию 10), режим построения (автоматически, OSTIS, локально), «Сохранить в файл» (TXT, HTML, DOCX, JSON, SCs), «Печать» и «Документ в sc-web».

- **1. Классический реферат** — предложения с наибольшим весом в порядке текста; под каждым — номер, абзац, вес = $\text{Score} \times \text{Posd} \times \text{Posp}$, место по весу и термины с наибольшим вкладом. Переключатель: «сжатые предложения» или «как в тексте».
- **2. Реферат в виде списка ключевых слов** — дерево: ключевое слово, под ним словосочетания, которые его содержат; полоса показывает значимость, число — вхождения в документ.
- **3. Как построен реферат** (раскрывается) — $|D|$, число предложений, терминов и $|DB|$; таблица весов терминов; таблица самых весомых предложений; время по этапам; в режиме OSTIS — узлы документа, реферата и действия в базе знаний и совпадение с локальным расчётом.

Пример. Статья о семантических моделях баз знаний: 18 192 знака, 85 предложений, 433 термина, $|DB| = 10$. Реферат — 10 предложений, 4 456 знаков (24,5 % документа). Самый частый термин — «БЗ» ($tf = 83 = tf_{\max}$, $df = 1$): $w = 0,5 \cdot (1 + 83/83) \cdot \ln(10/1) = 2,303$; «онтология» ($tf = 31$): $w = 0,5 \cdot (1 + 31/83) \cdot \ln 10 = 1,581$. Самое весомое предложение — № 17, перечень нерешённых проблем: вес $44,42 = \text{Score } 55,16 \times \text{Posd } 0,805 \times \text{Posp } 1,000$. Ключевые слова: «модель → семантическая модель; модель структуризации...», «система → компьютерная система; интеллектуальная система...», «область → предметная область». Из 6 ключевых слов авторов список полностью покрывает 5.

Как это работает. Метод sentence extraction из методички: вычислить веса слов, затем веса предложений и выбрать N самых весомых предложений в порядке текста.

Термины. Учитываются слова из букв; числа, стоп-слова, служебные обороты («в свою очередь») и слова чужого алфавита отбрасываются. Русские слова приводятся к лемме rymorphu3 («нейронных сетей» → «нейронный», «сеть»), немецкие — к основе стеммера Snowball («Datenbanken» → «datenbank»). **Вес термина** — модифицированный TF·IDF:

$$w(t, D) = 0,5 \cdot (1 + tf(t, D) / tf_{\max}(D)) \cdot \ln(|DB| / df(t))$$

где $tf(t, D)$ — частота термина в документе, $tf_{\max}(D)$ — наибольшая частота термина в документе, $df(t)$ — число документов с термином, $|DB|$ — число документов базы (документы коллекции того же языка). Множитель $0,5 \cdot (1 + tf/tf_{\max})$ сглаживает частоту, а логарифм обнуляет вес термина, который есть во всех документах. **Вес предложения** — произведение трёх функций:

$$\text{Score}(S_i) = \sum_t tf(t, S_i) \cdot w(t, D)$$

$$\text{Posd}(S_i) = 1 - BD(S_i) / |D|, \quad \text{Posp}(S_i) = 1 - BP(S_i) / |P|$$

$$\text{Weight}(S_i) = \text{Score}(S_i) \cdot \text{Posd}(S_i) \cdot \text{Posp}(S_i)$$

Score — сумма весов терминов предложения ($tf(t, S_i)$ — частота термина в предложении). Posd и Posp учитывают положение: $|D|$ и $|P|$ — длина документа и абзаца в символах, BD и BP — сколько символов стоит перед предложением в документе и в абзаце. Предложение в начале документа и абзаца получает множители, близкие к 1. Например, в документе «Кот ловит мышь. Кот спит.» второе предложение весомее по Score (1,229 против 1,014 — в нём редкий термин «спать»), но перед ним 16 символов из 25: $\text{Posd} = \text{Posp} = 0,36$, вес 0,159, и в реферат из одного предложения попадает первое.

Сжатие. Методичка разрешает трансформировать предложения; система убирает ссылки на литературу «[3]», отсылки «(рис. 2)» и русские вводные обороты в начале («Таким образом,»). У немецких предложений вводное слово не убирается: спрягаемый глагол должен стоять вторым.

Ключевые слова — 10 существительных с наибольшей значимостью $tf(t, D) \cdot \ln(|DB| / df(t))$, без сглаживания: иначе слово, встреченное трижды, весило бы почти как главное понятие статьи. Словосочетания, встреченные не реже двух раз: прилагательное + существительное с согласованием («нейронная сеть»), существительное + существительное в родительном падеже («база данных»), имя, отчество и фамилия («Евгений Онегин»); у немецкого текста — прилагательное + существительное («künstliche Neuronen») и сложные слова («Datenbanksystem» при «Datenbank»). Группа приводится к именительному падежу и подчиняется ключевому слову, которое содержит: «сеть → нейронная сеть».

3. Исходный документ /doc/<документ>/source

Активная ссылка на исходный документ, которую требует методичка. Показан полный текст с заголовками разделов; предложения реферата выделены цветом, на полях — их порядковый номер в реферате. При наведении на любое предложение видны его номер, вес с тремя сомножителями и место по весу. Флажок «показать вес всех предложений» включает тепловую карту: чем выше предложение по весу, тем темнее фон. Ссылки ведут обратно к реферату, на оригинал и на текст файлом.

Пример. «Die Leiden des jungen Werthers»: 18 505 знаков, 26 абзацев и 8 заголовков, 111 предложений, отмечено 10. Самое весомое — № 24 «Als er eines Tages vom Grafen C. aus einer adeligen Runde...»: вес $18,08 = 28,68 \times 0,827 \times 0,763$. Точка после «C.» не разрежала предложение — это инициал.

Как это работает. Текст делится на абзацы по пустым строкам. Абзац, отмеченный «# », или короткая строка без знака конца предложения — заголовок: он входит в документ D и занимает в нём место, но в реферат не попадает. Граница предложения —

«.», «!», «?» или «...», затем пробел и заглавная буква, цифра или открывающая кавычка. Граница не ставится после сокращений («т. е.», «рис.», «з. В.», «vgl.»), инициалов («А. С. Пушкин», «J. W. Goethe»), номеров пунктов («2.1.»), немецких порядковых числительных («im 19. Jahrhundert») и внутри прямой речи. Все смещения BD и BP считаются в той же строке D, что и на странице, поэтому подсветка точно совпадает с расчётом.

4. Свой документ /text

Поле для текста (абзацы — через пустую строку, заголовок можно отметить «# ») или загрузка файла: TXT, MD, HTML, DOCX, PDF до 8 МиБ. Можно задать заголовок, язык (по умолчанию — «определить по алфавиту») и число предложений (1–50). После «Построить реферат» открывается такая же страница реферата, как для документа коллекции, — с раскрытым разделом расчёта, сохранением, печатью и исходным документом. Документы сеанса перечислены под формой и на странице «Коллекция».

Пример. Текст из четырёх предложений о нейронных сетях, размер реферата 2: язык — русский, $|DB| = 11$, в реферат вошли предложения 1 и 3; ключевое слово «сеть → нейронная сеть». Термин «разметить», которого нет в коллекции, получил $df = 1$ и $\ln(11/1) = 2,398$ — наибольший возможный множитель.

Как это работает. Текст извлекается из файла (python-docx для DOCX, rypdf для PDF, BeautifulSoup для HTML). Язык определяется подсчётом букв: кириллических больше — русский, латинских — немецкий; у этой пары разные алфавиты, и этого достаточно. Документная частота $df(t)$ берётся по коллекции того же языка, а сам документ добавляется к базе: $|DB| = 10 + 1 = 11$, иначе у его новых терминов df был бы равен нулю и логарифм не определён. Дальше работают те же формулы, что и для документов коллекции.

5. Оценка /evaluation

Кнопка «Выполнить оценку» (около минуты) строит рефераты всех документов и сравнивает их с эталоном. На странице — таблицы полноты ROUGE-1 и ROUGE-2 по группам документов для метода и базовых способов отбора, точность P и F-мера метода, оценка реферата в виде ключевых слов, время на документ (локально и через OSTIS), зависимость от размера реферата (3–20 предложений), влияние выбора базы для $df(t)$ и таблица по документам. Полная выгрузка — report/evaluation.json.

Пример. Вся коллекция: $R_1 = 0,290$ и $R_2 = 0,070$ у метода против 0,229 и 0,062 у первых десяти предложений и 0,176 и 0,035 у случайных; потолок (оракул) — 0,455 и 0,153, то есть метод набирает 64 % потолка.

Лучше всего реферируются русские научные статьи ($R_1 = 0,479$, 72 % потолка), хуже всего — немецкие литературоведческие тексты (0,123 при потолке 0,272): их вводный раздел почти не совпадает по лексике с пересказом сюжета.

Ключевые слова покрывают 60,1 % ключевых слов авторов статей — вдвое больше, чем список самых частых существительных (29,1 %).

Время на документ: около 116 мс для русского и около 20 мс для немецкого текста; сами формулы — меньше 1 мс, остальное — разбор текста и морфология.

Как это работает. ROUGE-N сравнивает n-граммы реферата и эталона. N-граммы строятся по леммам значимых слов: ROUGE-1 — отдельные слова, ROUGE-2 — пары соседних слов.

$$R_N = \sum_g \min(c_{\text{реф}}(g), c_{\text{эт}}(g)) / \sum_g c_{\text{эт}}(g), \quad P_N = \sum_g \min(c_{\text{реф}}(g), c_{\text{эт}}(g)) / \sum_g c_{\text{реф}}(g)$$

$$F_N = 2 \cdot P_N \cdot R_N / (P_N + R_N)$$

где $c_{\text{реф}}(g)$ и $c_{\text{эт}}(g)$ — сколько раз n-грамма g встречается в реферате и в эталоне. Реферат из 10 предложений длиннее эталона, поэтому главная мера — полнота R . Для сравнения тот же объём отбирается базовыми способами: первые N предложений, случайные N (среднее по 30 попыткам), только Score, только $\text{Posd} \cdot \text{Posp}$. **Оракул** жадно, по одному, добавляет предложение, сильнее всего повышающее $R_1 + R_2$ по самому эталону, — это потолок для любого отбора предложений. Точность ключевых слов — доля ключевых слов верхнего уровня, встречающихся в эталоне; полнота — доля ключевых слов авторов, все слова которых вошли в список системы.

6. OSTIS /ostis

Три карточки: «Состояние» (подключено ли к sc-серверу `ws://localhost:8090`, кнопка «Подключиться заново»), «База знаний» (сколько документов коллекции в базе знаний, ссылки на sc-web и на раздел предметной области) и «sc-агент построения реферата» (сколько действий выполнено, последний документ и время). Ниже — файлы онтологии (пять файлов SCs, открываются по ссылке), схема работы «Как устроено», инструкция по запуску НИКА и журнал событий.

Пример. Загрузка онтологии — около 0,3 с при первом подключении; проверка и загрузка 20 документов в базу знаний — 130,5 мс. Вызов агента — в среднем 297,7 мс (из них работа агента — 147,5 мс, первый вызов для языка — 506,7 мс), чтение готового реферата из базы знаний — 69,3 мс. Рефераты агента совпали с локальным расчётом для 20 документов из 20.

Как это работает. Используется ostis-система НИКА (sc-machine 0.8.0) в контейнерах Docker; «Изборник» подключается к её sc-серверу по JSON через WebSocket (py-sc-client 0.3.0, py-sc-kpm 0.2.0). При подключении он загружает онтологию предметной области автоматического реферирования запросом `create_elements_by_scs` (если её ещё нет), помещает в базу знаний документы и регистрирует sc-агента.

- **Документ** — узел `doc_<идентификатор>` классов `concept_text_document`, `concept_test_collection_document` и класса предметной области; текст — sc-ссылка, связанная отношением `nrel_document_text` и принадлежащая языку `lang_ru` или `lang_de`.
- **Действие.** Интерфейс создаёт действие классов `question` и `action_build_summary` с аргументами `rrel_1` — документ и `rrel_2` — число предложений — и добавляет его в `question_initiated`. Появление этой дуги запускает sc-агента.
- **Агент** читает из базы знаний текст документа и тексты коллекции того же языка, строит реферат той же функцией, что и локальный режим, и одним запросом записывает около 500 sc-элементов: узел `concept_summary`, связанный с документом отношением `nrel_summary`, разделы `rrel_classic_summary` и `rrel_keyword_summary`, предложения — sc-ссылки, упорядоченные ролями `rrel_1`, `rrel_2`, ..., с числами

Score, Posd, Posp и веса. Затем связывает реферат с действием отношением nrel_answer и завершает действие (question_finished_successfully).

- **Повторный запрос** реферата того же размера читается из базы знаний без вызова агента. Если sc-сервер недоступен, режим «автоматически» строит реферат локально.